PARTE I

Guía de Estudio de la Asignatura

Planificación docente, competencias, metodología y cronograma semanal de trabajo

> Documento orientador que detalla la organización temporal de la materia a lo largo de las quince semanas lectivas. Para cada sesión presencial de teoría y laboratorio se establecen los objetivos de aprendizaje según la taxonomía de Bloom, los materiales de estudio requeridos y las recomendaciones de trabajo autónomo personal.

CC BY-SA 4.0 Internacional · Curso 2026-2027

Indice de contenidos

1 Introducción a la asignatura **2**

2 Datos identificativos y competencias **3**
2.1 Ficha técnica de la asignatura 3
2.2 Competencias generales y específicas 3

3 Metodología docente y distribución del trabajo **4**

4 Cronograma y plan de trabajo semanal **5**
4.1 BLOQUE I: Optimización Matemática y Ciencia de Datos (Semanas 1 a 10) 5
4.1.1 SEMANA 1: Fundamentos y Modelado Matemático de la Optimización . . . 5
4.1.2 SEMANA 2: Optimización No Lineal Sin Restricciones: Condiciones de Optimalidad 6
4.1.3 SEMANA 3: Algoritmos de Descenso y Visualización Avanzada 7
4.1.4 SEMANA 4: Análisis Convexo I: Conjuntos Convexos 8
4.1.5 SEMANA 5: Análisis Convexo II: Funciones Convexas 9
4.1.6 SEMANA 6: Optimización con Restricciones y Condiciones KKT 11
4.1.7 SEMANA 7: Dualidad Lagrangiana y Teoremas de Dualidad 12
4.1.8 SEMANA 8: Programación Convexa Disciplinada y CVXPY 13
4.1.9 SEMANA 9: Aplicaciones a Ciencia de Datos I: Regresión y SVM Lineal . . . 14
4.1.10 SEMANA 10: Aplicaciones a Ciencia de Datos II: SVM Generalizada, Dualidad y Kernels 15
4.2 BLOQUE II: Optimización y Análisis de Redes (Semanas 11 a 15) 16
4.2.1 SEMANA 11: Teoría de Grafos, Estructuras y Árboles Soporte 16
4.2.2 SEMANA 12: Algoritmos de Caminos Mínimos 17
4.2.3 SEMANA 13: Problemas de Flujo en Redes 18
4.2.4 SEMANA 14: Problemas de Emparejamiento (Matching) y Asignación 19
4.2.5 SEMANA 15: Rutas en Redes, TSP, VRP y Síntesis Final 21

5 Sistema de evaluación y criterios de calificación **23**
5.1 Convocatoria Ordinaria (Evaluación Continua) 23
5.2 Convocatoria Extraordinaria 23

6 Bibliografía y recursos de referencia **24**
6.1 Textos de referencia principal 24
6.2 Textos complementarios de algoritmos y computación 24
6.3 Recursos digitales y software 24

1 Introducción a la asignatura

La **optimización matemática** y el **análisis de redes** constituyen dos de los pilares fundamentales de la matemática aplicada, la investigación operativa y la ciencia de datos moderna. En la sociedad digital e industrial contemporánea, donde los recursos son finitos y los datos interconectados crecen de forma exponencial, la capacidad de formalizar matemáticamente problemas complejos y resolverlos algorítmicamente resulta esencial para la toma de decisiones estratégicas.

Esta asignatura, diseñada para el **Grado en Matemáticas** de la **Universidad Rey Juan Carlos (URJC)**, amplía y consolida la formación matemática previa en álgebra lineal, cálculo multivariante y programación lineal, adentrándose en el análisis riguroso de la **optimización no lineal continua** y el modelado discreto mediante la **teoría y optimización en redes**.

El diseño pedagógico del curso conjuga el rigor deductivo formal (teoremas de existencia, convexidad, dualidad lagrangiana y complejidad algorítmica) con el desarrollo computacional en **Python**, empleando librerías científicas profesionales de referencia internacional como **NumPy**, **SciPy**, **CVXPY** (para optimización convexa disciplinada) y **NetworkX** (para el modelado y visualización de grafos).

---

2 Datos identificativos y competencias

2.1. Ficha técnica de la asignatura

- **Asignatura**: Optimización y Análisis de Redes
- **Titulación**: Grado en Matemáticas (Mención en Matemática Aplicada y Computación) / Dobles Grados afines
- **Centro**: Escuela Técnica Superior de Ingeniería Informática (ETSII), Universidad Rey Juan Carlos
- **Créditos ECTS**: 6.0 ECTS (150 horas totales de dedicación del estudiante)
- **Carácter**: Asignatura optativa de cuarto curso (4.º curso)
- **Temporalidad**: Segundo cuatrimestre, 4.º curso (15 semanas lectivas)
- **Idioma de impartición**: Castellano
- **Prerrequisitos recomendados**: Álgebra Lineal, Cálculo Diferencial e Integral en Varias Variables, Fundamentos de Programación e Introducción a la Optimización Lineal.

2.2. Competencias generales y específicas

Al superar la asignatura, los estudiantes habrán adquirido las siguientes capacidades formativas:

1. **Modelización matemática formal**: Capacidad para abstraer problemas reales de logística, telecomunicaciones, finanzas y ciencia de datos, formulándolos rigurosamente como problemas de optimización matemática o modelos sobre redes.
2. **Análisis teórico y deductivo**: Habilidad para caracterizar la convexidad de conjuntos y funciones, demostrar condiciones de optimalidad de primer y segundo orden (KKT) y formular problemas duales interpretando sus propiedades de convergencia.
3. **Algoritmia y complejidad computacional**: Comprensión profunda de la estructura, convergencia y coste asintótico de los algoritmos clásicos y modernos de optimización en espacios continuos y sobre grafos finitos.
4. **Implementación y experimentación computacional**: Destreza para programar algoritmos numéricos y combinatorios en Python, validar soluciones con optimizadores comerciales y de código abierto, y comunicar resultados técnicos mediante visualizaciones claras.

---

3 Metodología docente y distribución del trabajo

La asignatura se articula en torno a un modelo de aprendizaje activo y equilibrado entre la fundamentación analítica y la experimentación computacional en laboratorio:

- **Clases de teoría y fundamentos matemáticos (2.0 horas/semana)**: Deducción formal de teoremas, análisis poliédrico, demostración de propiedades de convexidad, condiciones de Karush-Kuhn-Tucker y dualidad.
- **Sesiones de laboratorio en Python (2.0 horas/semana)**: Implementación práctica de algoritmos, resolución de modelos numéricos con CVXPY y NetworkX, y visualización de soluciones y superficies de nivel con Matplotlib.
- **Seminarios y resolución guiada de problemas**: Análisis paso a paso de casos de estudio, formulaciones complejas de programación entera y ejercicios de examen.
- **Tutorías académicas personalizadas**: Orientación individual y en grupo para la resolución de dudas teóricas y seguimiento de los proyectos de laboratorio.
- **Trabajo autónomo del estudiante (6.0 horas/semana)**: Estudio personal, lectura de la bibliografía recomendada, preparación de sesiones prácticas y resolución de colecciones de problemas.

Actividad formativa	Horas presenciales	Horas de trabajo autónomo	Horas totales
Clases de teoría magistral	30.0 h	35.0 h	65.0 h
Prácticas de laboratorio en Python	26.0 h	34.0 h	60.0 h
Seminarios y evaluación continua	4.0 h	21.0 h	25.0 h
Total dedicación (6 ECTS)	**60.0 h**	**90.0 h**	**150.0 h**

4 Cronograma y plan de trabajo semanal

El programa docente se estructura cronológicamente a lo largo de las 15 semanas lectivas del cuatrimestre, dividido en dos grandes bloques temáticos complementarios. Para cada sesión presencial de teoría y laboratorio se detallan los contenidos cubiertos, los objetivos didácticos formulados según la taxonomía de Bloom, las actividades planificadas, los materiales específicos a emplear y la recomendación cuantitativa de trabajo autónomo personal. Las referencias a secciones teóricas corresponden a la numeración interna del bloque de Apuntes Teóricos (Parte II), fácilmente accesibles a través del Índice General del volumen o mediante la búsqueda del tema correspondiente:

4.1. BLOQUE I: Optimización Matemática y Ciencia de Datos (Semanas 1 a 10)

4.1.1. SEMANA 1: Fundamentos y Modelado Matemático de la Optimización

Esta primera semana introduce el marco conceptual de la optimización matemática, sentando las bases analíticas del modelado formal y configurando el entorno de programación científica en Python.

Sesión 1.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Naturaleza de la optimización matemática, variables de decisión, parámetros del sistema, funciones objetivo y regiones factibles. Taxonomía de problemas (continuos frente a discretos, lineales frente a no lineales, convexos frente a no convexos). Repaso sistemático de álgebra lineal numérica, normas vectoriales y cálculo diferencial multivariante.
- *Objetivos de aprendizaje*:

  1. **Identificar** y **formular** matemáticamente los elementos constitutivos de un problema de optimización a partir de una descripción en lenguaje natural.
  2. **Clasificar** problemas de optimización atendiendo a su linealidad, diferenciabilidad y geometría admisible.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 1, "Fundamentos de la Optimización Matemática", Secciones 1.1 a 1.4.
  - **Diapositivas de clase**: Presentación del Tema 1, diapositivas 1 a 15.

- *Actividades planificadas*: Exposición magistral interactiva sobre modelización matemática formal y deducción de propiedades de normas vectoriales en pizarra.

- *Recomendación de trabajo autónomo (3.0 h)*: Lectura reflexiva y analítica del Tema 1 en los apuntes teóricos, centrando el estudio en las definiciones de normas, conjuntos de nivel y operadores diferenciales.

Sesión 1.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Configuración del entorno de desarrollo científico (`Anaconda`, entorno virtual `oar_env`, `VS Code` y `Jupyter`). Manipulación vectorial, cálculo matricial y álgebra lineal computacional con **NumPy**.
- *Objetivos de aprendizaje*:

  1. **Configurar** y **gestionar** entornos virtuales reproducibles para computación científica en Python.
  2. **Manipular** vectores, matrices, normas y gradientes numéricos de forma vectorizada y eficiente.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio1_fundamentos_numpy.ipynb` y script auxiliar `verificar_entorno.py`.
  - **Colección de ejercicios**: Bloque 1 de problemas (Ejercicios 1.1 a 1.6: formalización de modelos y cálculo multivariante).
  - **Entorno de software**: Python 3.11, NumPy, Jupyter Notebook.

- *Actividades planificadas*: Configuración guiada del entorno virtual del curso e implementación de rutinas numéricas vectorizadas con NumPy en el laboratorio.
- *Recomendación de trabajo autónomo (3.0 h)*: Finalización de los ejercicios prácticos del cuaderno de laboratorio 1 y resolución individual de los problemas 1.1 a 1.6 de la colección de ejercicios.

4.1.2. SEMANA 2: Optimización No Lineal Sin Restricciones: Condiciones de Optimalidad

Se profundiza en el análisis analítico de funciones diferenciables en varias variables, deduciendo las condiciones necesarias y suficientes de optimalidad y programando el método del descenso de gradiente.

Sesión 2.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Caracterización analítica de óptimos locales y globales en $\mathbb{R}^n$. Teorema de Fermat multivariante y condición necesaria de primer orden (puntos estacionarios). Condiciones necesarias y suficientes de segundo orden mediante la matriz Hessiana. Direcciones de descenso y ángulo respecto al vector gradiente.
- *Objetivos de aprendizaje*:

  1. **Calcular** puntos estacionarios de funciones diferenciables multivariantes.
  2. **Clasificar** extremos locales analizando el carácter definido de la matriz Hessiana.

- *Materiales específicos a emplear*:

- **Apuntes teóricos (Parte II)**: Tema 2, "Optimización No Lineal Sin Restricciones", Secciones 2.1 a 2.4.
- **Diapositivas de clase**: Presentación del Tema 2 (Parte I), diapositivas 1 a 35.

- *Actividades planificadas*: Deducción formal de las condiciones de segundo orden a partir del desarrollo de Taylor de orden 2 y discusión geométrica de la ortogonalidad del gradiente a las curvas de nivel.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio analítico de la demostración del Teorema de segundo orden y resolución manual de puntos críticos en funciones polinómicas y racionales.

Sesión 2.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Implementación algorítmica modular del método de **Descenso de Gradiente (Gradient Descent)**. Análisis empírico de la velocidad de convergencia, oscilaciones y sensibilidad a la tasa de aprendizaje (*learning rate*) en funciones cuadráticas mal condicionadas.
- *Objetivos de aprendizaje*:

  1. **Programar** el algoritmo de descenso de gradiente modular con criterios de parada por norma del gradiente y número máximo de iteraciones.
  2. **Evaluar** experimentalmente el impacto del número de condición de la matriz Hessiana sobre la trayectoria de convergencia.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio2_descenso_gradiente.ipynb`.
  - **Colección de ejercicios**: Bloque 2 de problemas (Ejercicios 2.1 a 2.5: cálculo de gradientes y clasificación espectral de Hessianos).
  - **Entorno de software**: Python 3.11, NumPy, Matplotlib.

- *Actividades planificadas*: Programación guiada del bucle iterativo de descenso de gradiente y experimentación con distintas tasas de aprendizaje en funciones elípticas.
- *Recomendación de trabajo autónomo (3.0 h)*: Completar el análisis de convergencia en el cuaderno de laboratorio 2 y resolver los problemas 2.1 a 2.5 del Bloque 2.

4.1.3. SEMANA 3: Algoritmos de Descenso y Visualización Avanzada

Esta semana se estudian los métodos de orden superior y las técnicas de búsqueda lineal inexacta, culminando con la visualización tridimensional y el uso de optimizadores de referencia en `scipy.optimize`.

Sesión 3.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Métodos de búsqueda lineal unidimensional inexacta (condición de descenso suficiente de Armijo y condiciones de curvatura de Wolfe). El **Método de Newton** en optimización multidimensional, velocidad de convergencia cuadrática local y coste de cálculo e inversión del Hessiano. Introducción a métodos cuasi-Newton (actualización BFGS) y direcciones conjugadas.
- *Objetivos de aprendizaje*:

1. **Comparar** la velocidad de convergencia lineal del gradiente frente a la convergencia cuadrática de Newton.
2. **Aplicar** las condiciones de Armijo y Wolfe para garantizar el descenso monótono y evitar pasos espurios.

- *Materiales específicos a emplear*:
  - **Apuntes teóricos (Parte II)**: Tema 2, "Optimización No Lineal Sin Restricciones", Secciones 2.5 a 2.8.
  - **Diapositivas de clase**: Presentación del Tema 2 (Parte II), diapositivas 36 a 74.
- *Actividades planificadas*: Demostración formal del paso de Newton, análisis de modificaciones para matrices indefinidas (regularización de Levenberg-Marquardt) y derivación de la actualización BFGS.
- *Recomendación de trabajo autónomo (3.0 h)*: Lectura reflexiva sobre métodos cuasi-Newton y deducción manual de un paso de Newton multidimensional.

Sesión 3.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Visualización avanzada de funciones de coste en dos dimensiones (curvas de nivel) y tres dimensiones (superficies de nivel) con trayectorias de descenso superpuestas en Matplotlib. Uso profesional de optimizadores en `scipy.optimize` (`minimize`).
- *Objetivos de aprendizaje*:
  1. **Generar** representaciones visuales avanzadas en 2D y 3D de paisajes de optimización no lineales.
  2. **Integrar** y **comparar** métodos numéricos (BFGS, CG, Nelder-Mead) resolviendo la función de Rosenbrock con `scipy.optimize`.
- *Materiales específicos a emplear*:
  - **Cuaderno y scripts de laboratorio**: `laboratorio3_newton_scipy.ipynb`.
  - **Colección de ejercicios**: Bloque 2 de problemas (Ejercicios 2.6 a 2.10: iteraciones de Newton y convergencia).
  - **Entorno de software**: Python 3.11, SciPy (`scipy.optimize`), Matplotlib.
- *Actividades planificadas*: Implementación de gráficos interactivos de curvas de nivel con trayectorias de optimizadores y benchmarking empírico de tiempos de cómputo.
- *Recomendación de trabajo autónomo (3.0 h)*: Redacción del informe comparativo de optimizadores en el cuaderno 3 y resolución de los ejercicios 2.6 a 2.10 del Bloque 2.

4.1.4. SEMANA 4: Análisis Convexo I: Conjuntos Convexos

Se introduce la geometría convexa como base matemática fundamental de la optimización moderna, estudiando la estructura de conjuntos, conos y los teoremas de separación.

Sesión 4.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Definición geométrica y algebraica de conjunto convexo. Combinaciones convexas y envoltura convexa (*convex hull*). Conos convexos y cono dual. Familias canónicas de conjuntos convexos (hiperplanos, semiespacios, poliedros, bolas euclídeas y elipsoides). Teoremas fundamentales del Hiperplano Separador y de Soporte. Operaciones que preservan convexidad.
- *Objetivos de aprendizaje*:
    1. **Demostrar** rigurosamente la convexidad de conjuntos matemáticos parametrizados.
    2. **Aplicar** los teoremas de separación para verificar analíticamente la disyunción entre regiones convexas.
- *Materiales específicos a emplear*:
    - **Apuntes teóricos (Parte II)**: Tema 3, "Análisis Convexo", Secciones 3.1 a 3.4.
    - **Diapositivas de clase**: Presentación del Tema 3 (Parte I), diapositivas 1 a 36.
- *Actividades planificadas*: Demostración analítica del Lema de Proyección sobre conjuntos convexos cerrados y deducción del Teorema del Hiperplano Separador estricto.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio detallado de las demostraciones de separación y caracterización analítica de conos duales de conjuntos poliédricos.

Sesión 4.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Representación y verificación computacional de poliedros definidos por intersección de semiespacios lineales. Cálculo de envolturas convexas en dos y tres dimensiones utilizando `scipy.spatial`.
- *Objetivos de aprendizaje*:
    1. **Calcular** y **visualizar** envolturas convexas a partir de nubes de puntos sintéticas y reales en Python.
    2. **Comprobar** la acotación y factibilidad geométrica de poliedros mediante álgebra lineal computacional.
- *Materiales específicos a emplear*:
    - **Cuaderno y scripts de laboratorio**: `laboratorio4_conjuntos_convexos.ipynb`.
    - **Colección de ejercicios**: Bloque 3 de problemas (Ejercicios 3.1 a 3.6: demostraciones de conjuntos convexos y separación).
    - **Entorno de software**: Python 3.11, SciPy (`scipy.spatial`), NumPy, Matplotlib.
- *Actividades planificadas*: Cálculo práctico de envolturas convexas tridimensionales con visualización interactiva y comprobación de pertenencia de puntos a poliedros.
- *Recomendación de trabajo autónomo (3.0 h)*: Finalización de los ejercicios del cuaderno 4 y resolución completa de los problemas 3.1 a 3.6 de la colección de ejercicios.

4.1.5. SEMANA 5: Análisis Convexo II: Funciones Convexas

Se abordan las funciones convexas, sus caracterizaciones diferenciales y el resultado central que garantiza la coincidencia entre óptimos locales y globales.

Sesión 5.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Definición formal de función convexa, estrictamente convexa y fuertemente convexa. Caracterización por epígrafe (*epigraph*) y conjuntos de subnivel. Condiciones diferenciales de primer orden (subgradientes y plano tangente inferior). Condiciones de segundo orden: semidefinición positiva de la matriz Hessiana ($\nabla^2 f(x) \succeq 0$). Teorema fundamental de equivalencia entre mínimos locales y globales. Desigualdad de Jensen.
- *Objetivos de aprendizaje*:

  1. **Caracterizar** la convexidad de funciones escalares y multivariantes mediante criterios diferenciales de primer y segundo orden.
  2. **Demostrar** que en un problema de optimización convexa todo mínimo local es automáticamente mínimo global.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 3, "Análisis Convexo", Secciones 3.5 a 3.8.
  - **Diapositivas de clase**: Presentación del Tema 3 (Parte II), diapositivas 37 a 70.

- *Actividades planificadas*: Demostración rigurosa del teorema de equivalencia entre óptimos locales y globales, análisis de operaciones que preservan convexidad de funciones y prueba de la desigualdad de Jensen.
- *Recomendación de trabajo autónomo (3.0 h)*: Asimilación de las reglas de composición convexa (composición atomística) y cálculo analítico de subgradientes en funciones no diferenciables.

Sesión 5.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Verificación numérica continua de convexidad mediante análisis espectral de autovalores del Hessiano. Verificación computacional y empírica de la desigualdad de Jensen en distribuciones muestrales.
- *Objetivos de aprendizaje*:

  1. **Implementar** rutinas computacionales en Python para evaluar la semidefinición positiva de matrices hessianas sobre dominios continuos.
  2. **Simular** empíricamente el cumplimiento de la desigualdad de Jensen y trazar epígrafes de funciones convexas.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio5_funciones_convexas.ipynb`.
  - **Colección de ejercicios**: Bloque 3 de problemas (Ejercicios 3.7 a 3.12: funciones convexas, Hessianos y desigualdades).
  - **Entorno de software**: Python 3.11, NumPy, SciPy, Matplotlib.

- *Actividades planificadas*: Programación de pruebas espectrales de matrices y trazado de curvas de subnivel de funciones cuadráticas y racionales.
- *Recomendación de trabajo autónomo (3.0 h)*: Completar las conclusiones del cuaderno de laboratorio 5 y resolver los ejercicios 3.7 a 3.12 de la colección de problemas.

4.1.6. SEMANA 6: Optimización con Restricciones y Condiciones KKT

Se estudia la teoría de la optimización con restricciones no lineales de igualdad y desigualdad, deduciendo las condiciones de Karush-Kuhn-Tucker (KKT).

Sesión 6.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Formulación estándar de problemas con restricciones de igualdad y desigualdad. Geometría de las restricciones y conos de direcciones factibles y de descenso. Cualificación de restricciones: condición de independencia lineal de gradientes (LICQ) y condición de Mangasarian-Fromovitz (MFCQ). Teorema de las **condiciones de Karush-Kuhn-Tucker (KKT)**: primal factibilidad, dual factibilidad, holgura complementaria y estacionariedad. Suficiencia de KKT bajo convexidad.
- *Objetivos de aprendizaje*:

  1. **Formular** y **resolver** analíticamente el sistema algebraico de condiciones KKT.
  2. **Verificar** la cualificación de restricciones en puntos frontera comprobando la condición LICQ.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 4, "Optimización con Restricciones y Dualidad", Secciones 4.1 a 4.4.
  - **Diapositivas de clase**: Presentación del Tema 4 (Parte I), diapositivas 1 a 38.

- *Actividades planificadas*: Deducción formal de las condiciones KKT mediante conos polares y el Lema de Farkas, con análisis geométrico de restricciones activas e inactivas.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio analítico de la demostración de suficiencia de KKT bajo hipótesis de convexidad y resolución manual de sistemas KKT con restricciones cuadráticas.

Sesión 6.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Resolución analítica asistida por ordenador de sistemas KKT no lineales con cálculo simbólico (`sympy`) y numérico. Visualización gráfica en 2D de conos de gradientes de restricciones activas y vectores de multiplicadores de Lagrange.
- *Objetivos de aprendizaje*:

  1. **Resolver** sistemas algebraicos no lineales de KKT mediante herramientas de cálculo computacional.
  2. **Interpretar** geométricamente el papel de los multiplicadores de Lagrange ortogonales a la frontera activa.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio6_condiciones_kkt.ipynb`.
  - **Colección de ejercicios**: Bloque 4 de problemas (Ejercicios 4.1 a 4.6: resolución detallada de sistemas KKT).
  - **Entorno de software**: Python 3.11, SymPy (cálculo simbólico), NumPy, Matplotlib.

- *Actividades planificadas*: Resolución de problemas cuadráticos con restricciones lineales en SymPy y trazado 2D de las curvas de nivel junto con las direcciones de las restricciones activas.

- *Recomendación de trabajo autónomo (3.0 h)*: Finalización de los ejercicios del cuaderno 6 y realización de los problemas 4.1 a 4.6 del Bloque 4.

4.1.7. SEMANA 7: Dualidad Lagrangiana y Teoremas de Dualidad

Se introduce el marco de la dualidad lagrangiana, el salto de dualidad, las condiciones de Slater y el análisis de sensibilidad mediante precios sombra.

Sesión 7.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Construcción formal de la función Lagrangiana y de la función dual de Lagrange $g(\lambda, \mu)$. Concavidad intrínseca de la función dual. Formulación del problema Dual de Lagrange. Propiedad de **Dualidad Débil** y salto de dualidad (*duality gap*). Teoremas de **Dualidad Fuerte** y condición de cualificación de Slater para problemas convexos. Interpretación económica y análisis de sensibilidad de los multiplicadores duales (precios sombra).
- *Objetivos de aprendizaje*:
  1. **Derivar** analíticamente la función dual de Lagrange asociada a un problema primal dado.
  2. **Evaluar** la existencia de dualidad fuerte mediante la verificación de la condición de Slater.
- *Materiales específicos a emplear*:
  - **Apuntes teóricos (Parte II)**: Tema 4, "Optimización con Restricciones y Dualidad", Secciones 4.5 a 4.8.
  - **Diapositivas de clase**: Presentación del Tema 4 (Parte II), diapositivas 39 a 72.
- *Actividades planificadas*: Demostración formal del teorema de dualidad débil y deducción geométrica del salto dual mediante conjuntos epígrafes de perturbación.
- *Recomendación de trabajo autónomo (3.0 h)*: Asimilación del concepto de precio sombra como derivada direccional del valor óptimo respecto al término independiente de las restricciones.

Sesión 7.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Construcción numérica y trazado computacional de la función dual de Lagrange en problemas cuadráticos. Verificación empírica de la dualidad fuerte y convergencia primal-dual.
- *Objetivos de aprendizaje*:
  1. **Trazar** numéricamente funciones duales cóncavas a partir de la minimización del lagrangiano.
  2. **Verificar** computacionalmente la convergencia de los valores óptimos primal y dual en problemas convexos.
- *Materiales específicos a emplear*:
  - **Cuaderno y scripts de laboratorio**: `laboratorio7_dualidad_lagrangiana.ipynb`.
  - **Colección de ejercicios**: Bloque 4 de problemas (Ejercicios 4.7 a 4.12: funciones duales y análisis de sensibilidad).
  - **Entorno de software**: Python 3.11, NumPy, SciPy, Matplotlib.

- *Actividades planificadas*: Experimentación con perturbaciones de las restricciones para validar numéricamente la relación entre multiplicadores duales y tasas de variación del óptimo.
- *Recomendación de trabajo autónomo (3.0 h)*: Completar el cuaderno de laboratorio 7 y resolver los problemas 4.7 a 4.12 de la colección de ejercicios.

4.1.8. SEMANA 8: Programación Convexa Disciplinada y CVXPY

Se estudia el modelado computacional formal de la optimización convexa, presentando la jerarquía de problemas canónicos y la librería declarativa CVXPY.

Sesión 8.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Jerarquía estructural de problemas convexos (Programación Lineal LP, Programación Cuadrática QP, Programación Cuadrática con Restricciones Cuadráticas QCQP y Conos de Segundo Orden SOCP). Principios formales de la **Programación Convexa Disciplinada (DCP)**: reglas de composición atomística, monotonicidad y análisis de curvatura.
- *Objetivos de aprendizaje*:

  1. **Clasificar** problemas convexos dentro de la jerarquía canónica de programación matemática.
  2. **Modelar** problemas respetando estrictamente el conjunto de reglas de la programación convexa disciplinada.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 4, "Optimización con Restricciones y Dualidad", Secciones 4.9 a 4.12.
  - **Diapositivas de clase**: Presentación del Tema 4 (Parte III), diapositivas 73 a 106.

- *Actividades planificadas*: Exposición sobre grafos de computación en DCP, transformaciones epígrafes y conversión a formas cónicas estándar para solvers interiores.
- *Recomendación de trabajo autónomo (3.0 h)*: Consulta de la documentación oficial de CVXPY y análisis de las reglas de composición de funciones convexas no decrecientes.

Sesión 8.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Introducción a la librería **CVXPY**. Declaración formal de variables, parámetros, restricciones y funciones objetivo. Resolución de problemas LP y QP, inspección de estados de convergencia del solver y recuperación de soluciones primales y multiplicadores duales.
- *Objetivos de aprendizaje*:

  1. **Traducir** modelos matemáticos declarativos a código ejecutable en CVXPY respetando las reglas DCP.
  2. **Interpretar** y **recuperar** certificados de optimalidad y valores de variables duales devueltos por el optimizador.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio8_cvxpy_dcp.ipynb`.

- **Colección de ejercicios**: Bloque 4 de problemas (Ejercicios 4.13 a 4.18: formulación matricial canónica de LP, QP y SOCP).
- **Entorno de software**: Python 3.11, CVXPY (solvers ECOS, OSQP, Clarabel).

- *Actividades planificadas*: Taller interactivo de formulación en CVXPY resolviendo problemas de optimización de carteras de inversión de Markowitz con restricciones de varianza.
- *Recomendación de trabajo autónomo (3.0 h)*: Finalización de los ejercicios del cuaderno 8 y realización de los problemas 4.13 a 4.18 del Bloque 4.

4.1.9. SEMANA 9: Aplicaciones a Ciencia de Datos I: Regresión y SVM Lineal

Se conectan los fundamentos de la optimización convexa con el aprendizaje automático, estudiando la regresión regularizada y las máquinas de vectores de soporte de margen duro.

Sesión 9.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Formulación matricial de la regresión lineal por mínimos cuadrados ordinarios (OLS). Regularización y selección de variables: regresión Ridge (L_2) y regresión Lasso (L_1). Fundamentos del aprendizaje estadístico supervisado: clasificación binaria e hiperplanos separadores. **Máquinas de Vectores de Soporte (SVM)**: formulación matemática de margen máximo para datos linealmente separables (margen duro).
- *Objetivos de aprendizaje*:

  1. **Formular** problemas canónicos de regresión y clasificación como optimización convexa cuadrática.
  2. **Diferenciar** el efecto de la regularización L_1 inductora de dispersión frente a la regularización L_2 de contracción suave.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 5, "Aplicaciones a Ciencia de Datos y Aprendizaje Automático", Secciones 5.1 a 5.4.
  - **Diapositivas de clase**: Presentación del Tema 5 (Parte I), diapositivas 1 a 40.

- *Actividades planificadas*: Deducción matemática de la relación entre el ancho geométrico del margen ($2/\|w\|$) y el problema cuadrático primal de SVM en pizarra.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio analítico de la deducción de la solución cerrada de Ridge y de la no diferenciabilidad en el origen de la regularización Lasso.

Sesión 9.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Implementación y comparación de modelos de regresión OLS, Ridge y Lasso en CVXPY. Formulación y resolución del clasificador SVM lineal de margen duro en CVXPY. Visualización gráfica del hiperplano de separación, márgenes y vectores de soporte.
- *Objetivos de aprendizaje*:

  1. **Programar** modelos de regresión regularizada y clasificación lineal de margen duro en CVXPY.

2. **Identificar** gráfica y numéricamente los vectores de soporte críticos que definen la frontera de decisión.

- *Materiales específicos a emplear*:
  - **Cuaderno y scripts de laboratorio**: `laboratorio9_regresion_svm_duro.ipynb`.
  - **Colección de ejercicios**: Bloque 5 de problemas (Ejercicios 5.1 a 5.5: formulaciones de regularización y margen máximo).
  - **Entorno de software**: Python 3.11, CVXPY, NumPy, Matplotlib, Scikit-Learn (generación de datasets sintéticos).
- *Actividades planificadas*: Práctica de clasificación sobre conjuntos de datos sintéticos y trazado 2D de hiperplanos de margen máximo en Matplotlib.
- *Recomendación de trabajo autónomo (3.0 h)*: Completar el análisis de regularización en el cuaderno 9 y resolver los ejercicios 5.1 a 5.5 del Bloque 5.

4.1.10. SEMANA 10: Aplicaciones a Ciencia de Datos II: SVM Generalizada, Dualidad y Kernels

Se concluye el Bloque I con el clasificador SVM de margen blando, su formulación dual y la extensión a fronteras no lineales mediante el truco del Kernel.

Sesión 10.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Clasificación con solapamiento: formulación de margen blando mediante variables de holgura ξ_i. Hiperparámetro de penalización C y compromiso sesgo-varianza. Formulación dual de SVM con multiplicadores de Lagrange α_i en una caja $[0, C]$. El **truco del Kernel (Kernel Trick)**: funciones de base radial (RBF) y núcleos polinomiales en espacios de Hilbert de dimensión infinita. Teorema de Mercer.
- *Objetivos de aprendizaje*:
  1. **Derivar** rigurosamente el problema dual de las Máquinas de Vectores de Soporte a partir de las condiciones KKT del problema primal con holguras.
  2. **Aplicar** transformaciones por núcleos para resolver problemas de clasificación no linealmente separables.
- *Materiales específicos a emplear*:
  - **Apuntes teóricos (Parte II)**: Tema 5, "Aplicaciones a Ciencia de Datos y Aprendizaje Automático", Secciones 5.5 a 5.8.
  - **Diapositivas de clase**: Presentación del Tema 5 (Parte II), diapositivas 41 a 82.
- *Actividades planificadas*: Demostración de las condiciones de holgura complementaria en el dual de SVM y cálculo explícito de matrices de Gram para kernels RBF.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio reflexivo sobre la clasificación de puntos (interiores, de margen y violaciones) a partir del valor de sus multiplicadores duales α_i.

Sesión 10.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Resolución del problema dual de SVM no lineal en CVXPY. Benchmarking comparativo de precisión y tiempo de ejecución frente a `scikit-learn` (`sklearn.svm.SVC`). Validación cruzada y ajuste de hiperparámetros (C, γ).
- *Objetivos de aprendizaje*:

  1. **Implementar** el clasificador dual de SVM con kernels en CVXPY.
  2. **Contrastar** la formulación matemática directa en CVXPY con librerías optimizadas de producción en ciencia de datos.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio10_svm_dual_kernels.ipynb`.
  - **Colección de ejercicios**: Bloque 5 de problemas (Ejercicios 5.6 a 5.10: deducción del dual de SVM y matrices de Gram).
  - **Entorno de software**: Python 3.11, CVXPY, Scikit-Learn, Matplotlib.

- *Actividades planificadas*: Taller de ajuste de hiperparámetros sobre conjuntos de datos no separables con fronteras circulares y espirales.
- *Recomendación de trabajo autónomo (3.0 h)*: Entrega del cuaderno 10, resolución de los problemas 5.6 a 5.10 del Bloque 5 y repaso integral de los temas 1 a 5 para el Examen Parcial 1.

4.2. BLOQUE II: Optimización y Análisis de Redes (Semanas 11 a 15)

4.2.1. SEMANA 11: Teoría de Grafos, Estructuras y Árboles Soporte

Se inicia el Bloque II adentrándose en la teoría de redes y grafos, sus representaciones matriciales y los árboles soporte de mínimo peso.

Sesión 11.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Elementos formales de teoría de grafos: grafos no dirigidos $\{u, v\}$, digrafos ordenados (u, v), multigrafos y grafos bipartitos. Representaciones algebraicas: matriz de adyacencia, matriz de incidencia y matriz Laplaciana. Conectividad: caminos, componentes conexas, conjuntos de corte y árboles. **Árboles Soporte de Mínimo Peso (MST)**: algoritmo voraz de Kruskal y algoritmo de Prim. Demostración de corrección mediante la propiedad fundamental del corte.
- *Objetivos de aprendizaje*:

  1. **Representar** formalmente redes y topologías complejas mediante matrices algebraicas (adyacencia, incidencia y Laplaciana).
  2. **Calcular** el árbol generador de coste mínimo aplicando con rigor los algoritmos de Kruskal y Prim.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 6, "Introducción a la Teoría de Redes y Grafos", Secciones 6.1 a 6.5.

- **Diapositivas de clase**: Presentación del Tema 6, diapositivas 1 a 55.

- *Actividades planificadas*: Demostración analítica de la propiedad del corte para árboles generadores y análisis del espectro de la matriz Laplaciana en la conectividad algebraica.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio analítico de la prueba de corrección de Kruskal y deducción de la complejidad con estructuras Union-Find.

Sesión 11.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Modelado, construcción y manipulación de topologías de red con **NetworkX**. Cálculo de métricas estructurales (distribución de grados, coeficientes de agrupamiento, densidades y componentes conexas). Implementación y comparación de algoritmos MST sobre grafos sintéticos y redes geográficas reales.
- *Objetivos de aprendizaje*:

  1. **Utilizar** la librería NetworkX para la manipulación y análisis cuantitativo de redes complejas en Python.
  2. **Calcular** y **visualizar** árboles generadores de coste mínimo sobre redes de infraestructura real.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio11_grafos_networkx_mst.ipynb`.
  - **Colección de ejercicios**: Bloque 6 de problemas (Ejercicios 6.1 a 6.8: propiedades de árboles y cálculo manual de MST).
  - **Entorno de software**: Python 3.11, NetworkX, Matplotlib.

- *Actividades planificadas*: Construcción interactiva de grafos aleatorios y redes de transporte en NetworkX y trazado visual del árbol soporte mínimo superpuesto.
- *Recomendación de trabajo autónomo (3.0 h)*: Finalización del cuaderno 11 y resolución de los ejercicios 6.1 a 6.8 de la colección de problemas.

4.2.2. SEMANA 12: Algoritmos de Caminos Mínimos

Se abordan los algoritmos clásicos y eficientes de caminos mínimos en grafos valorados y sus aplicaciones a problemas de enrutamiento y finanzas.

Sesión 12.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: El Problema del Camino Mínimo (*Shortest Path Problem*) y principio de subestructura óptima de Bellman. **Algoritmo de Dijkstra** para redes con pesos no negativos e implementación con montículos binarios. **Algoritmo de Bellman-Ford**: tratamiento de pesos negativos y detección de ciclos absorbentes de coste negativo. **Algoritmo de Floyd-Warshall**: caminos mínimos entre todos los pares de vértices mediante programación dinámica. Caminos en digrafos acíclicos dirigidos (DAGs) y ordenación topológica.
- *Objetivos de aprendizaje*:

  1. **Seleccionar** y **aplicar** el algoritmo de caminos mínimos óptimo en función de la topología de la red y del signo de los costes.

2. **Modelar** problemas no espaciales (arbitraje financiero, planificación temporal) como búsquedas de caminos mínimos.

- *Materiales específicos a emplear*:
  - **Apuntes teóricos (Parte II)**: Tema 7, "Caminos Mínimos en Redes", Secciones 7.1 a 7.6.
  - **Diapositivas de clase**: Presentación del Tema 7, diapositivas 1 a 48.
- *Actividades planificadas*: Deducción formal de las ecuaciones de Bellman y demostración analítica de la corrección de los algoritmos de Dijkstra y Floyd-Warshall.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio reflexivo sobre la detección de ciclos de coste negativo y análisis de complejidad temporal y espacial en grafos densos y dispersos.

Sesión 12.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Implementación y análisis comparativo de eficiencia entre Dijkstra y Bellman-Ford en NetworkX. Detección de oportunidades de arbitraje en mercados financieros modeladas como ciclos de coste negativo sobre grafos dirigidos de tipos de cambio. Visualización de rutas geodésicas sobre redes viarias.
- *Objetivos de aprendizaje*:
  1. **Programar** detectores de ciclos negativos aplicando transformaciones logarítmicas de tipos de cambio en Python.
  2. **Trazar** visualmente caminos geodésicos sobre grafos dirigidos y mapas reales con NetworkX.
- *Materiales específicos a emplear*:
  - **Cuaderno y scripts de laboratorio**: `laboratorio12_caminos_minimos.ipynb`.
  - **Colección de ejercicios**: Bloque 7 de problemas (Ejercicios 7.1 a 7.8: trazado paso a paso de Dijkstra, Bellman-Ford y Floyd-Warshall).
  - **Entorno de software**: Python 3.11, NetworkX, Matplotlib.
- *Actividades planificadas*: Implementación de la rutina de arbitraje financiero en divisas y cálculo computacional de matrices de distancias de todos los pares.
- *Recomendación de trabajo autónomo (3.0 h)*: Completar el análisis de arbitraje en el cuaderno 12 y resolver los ejercicios 7.1 a 7.8 del Bloque 7.

4.2.3. SEMANA 13: Problemas de Flujo en Redes

Se analizan los modelos de redes capacitadas, el teorema fundamental de corte mínimo y los flujos a coste mínimo.

Sesión 13.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Redes de transporte con capacidades: leyes de conservación de flujo de Kirchhoff y redes residuales. **Problema del Flujo Máximo**: algoritmos de caminos de aumento (Ford-Fulkerson y Edmonds-Karp). Cortes en redes y capacidad de un corte.

Teorema del Flujo Máximo - Corte Mínimo (Max-Flow Min-Cut). **Problema del Flujo a Coste Mínimo (Min-Cost Flow)** y formulación de programación lineal. Teorema de integridad de flujos y matriz de incidencia totalmente unimodular.

- *Objetivos de aprendizaje*:

  1. **Calcular** la capacidad máxima de transporte de una red determinando el corte mínimo saturado que actúa como cuello de botella.
  2. **Demostrar** la total unimodularidad de la matriz de incidencia de digrafos y la integridad inherente de las soluciones.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 8, "Flujos en Redes", Secciones 8.1 a 8.6.
  - **Diapositivas de clase**: Presentación del Tema 8, diapositivas 1 a 45.

- *Actividades planificadas*: Demostración rigurosa del Teorema Max-Flow Min-Cut y desarrollo en pizarra del etiquetado de vértices y actualización de redes residuales.
- *Recomendación de trabajo autónomo (3.0 h)*: Asimilación de la prueba de convergencia polinomial $O(VE^2)$ de Edmonds-Karp y formulación algebraica de modelos de transbordo.

Sesión 13.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Resolución de problemas de flujo máximo y corte mínimo con solvers de NetworkX (`maximum_flow`, `minimum_cut`). Modelado de redes de abastecimiento y distribución logística con capacidades y costes lineales en CVXPY y NetworkX.
- *Objetivos de aprendizaje*:

  1. **Formular** y **resolver** problemas de flujo a coste mínimo y transporte multiescalón mediante optimización lineal en CVXPY.
  2. **Visualizar** redes residuales, arcos saturados y cuellos de botella en NetworkX.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio13_flujos_redes.ipynb`.
  - **Colección de ejercicios**: Bloque 8 de problemas (Ejercicios 8.1 a 8.8: resolución tabular de Ford-Fulkerson y formulación de transbordos).
  - **Entorno de software**: Python 3.11, NetworkX, CVXPY.

- *Actividades planificadas*: Simulación interactiva de cuellos de botella en redes de suministro logístico y contraste de soluciones entre NetworkX y CVXPY.
- *Recomendación de trabajo autónomo (3.0 h)*: Entrega de los ejercicios del cuaderno 13 y resolución de los problemas 8.1 a 8.8 de la colección de problemas.

4.2.4. SEMANA 14: Problemas de Emparejamiento (Matching) y Asignación

Se estudia la teoría combinatoria de emparejamientos en grafos bipartitos y generales, desde el Algoritmo Húngaro hasta las flores de Edmonds.

Sesión 14.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Emparejamientos en grafos: emparejamiento maximal, máximo y perfecto. Emparejamiento en grafos bipartitos: Teorema del Matrimonio de Hall y Teorema de Kőnig. El **Algoritmo Húngaro** (Kuhn-Munkres) para el problema clásico de asignación de tareas con costes. Emparejamiento en grafos generales no bipartitos: **Algoritmo de las Flores de Edmonds** (*Blossom Algorithm*) y contracción de ciclos impares. Teorema de estructura y descomposición de Gallai-Edmonds.
- *Objetivos de aprendizaje*:

  1. **Formular** problemas de asignación de recursos, turnos y personal como emparejamientos óptimos en grafos bipartitos y generales.
  2. **Comprender** el mecanismo de contracción de flores (*blossoms*) para resolver emparejamientos en grafos generales en tiempo polinomial.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 9, "Emparejamientos y Asignación en Grafos", Secciones 9.1 a 9.6.
  - **Diapositivas de clase**: Presentación del Tema 9, diapositivas 1 a 42.

- *Actividades planificadas*: Deducción del teorema de Kőnig a partir de flujos en redes, interpretación dual del Algoritmo Húngaro como potenciales de nodo y análisis de caminos alternantes.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio de la demostración del Teorema de Hall y análisis del papel de los ciclos alternantes impares en grafos no bipartitos.

Sesión 14.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Resolución de problemas de emparejamiento bipartito y asignación ponderada en NetworkX y `scipy.optimize.linear_sum_assignment`. Implementación y experimentación del algoritmo de Edmonds para matching máximo general sobre grafos arbitrarios.
- *Objetivos de aprendizaje*:

  1. **Resolver** computacionalmente problemas de emparejamiento óptimo ponderado y cardinal con librerías científicas en Python.
  2. **Aplicar** modelos de matching a casos reales (asignación de residentes médicos y programas de donación cruzada de órganos).

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio14_matching_asignacion.ipynb`.
  - **Colección de ejercicios**: Bloque 9 de problemas (Ejercicios 9.1 a 9.7: cálculo manual del algoritmo Húngaro y contracción de flores).
  - **Entorno de software**: Python 3.11, NetworkX, SciPy (`scipy.optimize`), Matplotlib.

- *Actividades planificadas*: Implementación de la asignación óptima de tareas y visualización gráfica de flores contraídas y caminos aumentantes en NetworkX.
- *Recomendación de trabajo autónomo (3.0 h)*: Finalización del cuaderno 14 y resolución de los problemas 9.1 a 9.7 del Bloque 9.

4.2.5. SEMANA 15: Rutas en Redes, TSP, VRP y Síntesis Final

Se cierra el curso con los problemas clásicos de diseño de rutas en redes, abordando la frontera entre problemas polinomiales y NP-duros.

Sesión 15.1: Teoría y Fundamentos Matemáticos (2.0 h presenciales)

- *Contenidos cubiertos*: Recorridos eulerianos: Teorema de Euler-Hierholzer y **Problema del Cartero Chino (CPP)** resuelto mediante eulerización por matching de mínimo peso. Ciclos hamiltonianos: complejidad $\mathcal{NP}$-completa, condiciones necesarias (2-conexidad) y condiciones suficientes (teoremas de Dirac y Ore, clausura de Bondy-Chvátal). **Problema del Viajante de Comercio (TSP)**: formulaciones de programación lineal entera (etapas x_{ijk}, eliminación de subtours DFJ y formulación compacta MTZ). Heurísticas de búsqueda local (2-Opt) y **Algoritmo de aproximación de Christofides** con factor 1.5 en grafos métricos. Introducción a los **Problemas de Rutas de Vehículos (VRP)**.
- *Objetivos de aprendizaje*:

  1. **Diferenciar** con precisión la frontera de complejidad computacional entre problemas eulerianos (polinomiales) y hamiltonianos ($\mathcal{NP}$-duros).
  2. **Formular** modelos de programación lineal entera para diseño de rutas óptimas seleccionando la técnica adecuada de eliminación de subtours.
  3. **Demostrar** la cota de aproximación 3/2 del algoritmo de Christofides sobre grafos métricos.

- *Materiales específicos a emplear*:

  - **Apuntes teóricos (Parte II)**: Tema 10, "Rutas en Redes: Recorridos Eulerianos, Hamiltonianos y Rutas de Vehículos", Secciones 10.1 a 10.7.
  - **Diapositivas de clase**: Presentación del Tema 10, diapositivas 1 a 52.

- *Actividades planificadas*: Demostración analítica de la cota de Christofides combinando árboles soporte mínimos y matchings perfectos, y análisis de la formulación entera MTZ en pizarra.
- *Recomendación de trabajo autónomo (3.0 h)*: Estudio de la formulación compacta MTZ y repaso comparativo general de todas las estructuras y algoritmos estudiados a lo largo del curso.

Sesión 15.2: Laboratorio Computacional y Resolución Práctica (2.0 h presenciales)

- *Contenidos cubiertos*: Implementación del algoritmo del Cartero Chino con NetworkX. Programación de la formulación compacta MTZ en CVXPY y resolución con solvers de programación entera mixta (MIP). Implementación de la heurística de mejora 2-Opt y del algoritmo de aproximación de Christofides.
- *Objetivos de aprendizaje*:

  1. **Implementar** modelos de programación entera mixta y heurísticas con garantías formales para problemas de transporte a gran escala en Python.
  2. **Resolver** instancias de rutas de recogida de residuos urbanos y distribución logística comparando tiempos y calidad de soluciones.

- *Materiales específicos a emplear*:

  - **Cuaderno y scripts de laboratorio**: `laboratorio15_rutas_tsp_vrp.ipynb`.

- **Colección de ejercicios**: Bloque 10 de problemas (Ejercicios 10.1 a 10.8: eulerización, formulaciones MTZ y algoritmo de Christofides).
- **Entorno de software**: Python 3.11, NetworkX, CVXPY (solvers MIP como GLPK/CBC o SCIP), Matplotlib.

- *Actividades planificadas*: Taller de síntesis computacional resolviendo problemas de rutas en grafos geográficos reales y benchmarking entre solvers exactos y heurísticas 2-Opt.
- *Recomendación de trabajo autónomo (3.0 h)*: Entrega del cuaderno 15, resolución de los problemas 10.1 a 10.8 del Bloque 10 y preparación integral para el Examen Parcial 2.

---

5 Sistema de evaluación y criterios de calificación

El sistema de evaluación está diseñado para valorar de forma continua tanto la solidez teórica y el rigor formal como la competencia computacional en la resolución de problemas prácticos:

5.1. Convocatoria Ordinaria (Evaluación Continua)

La calificación final de la convocatoria ordinaria se obtiene a partir de tres componentes ponderados:

1. **Examen Parcial 1 (Bloque I - Optimización Matemática y Ciencia de Datos)**: **35 % de la nota final**.
    - Prueba escrita individual compuesta por cuestiones teórico-deductivas (demostraciones de convexidad, KKT, dualidad) y resolución analítica de problemas numéricos.
2. **Examen Parcial 2 (Bloque II - Optimización y Análisis de Redes)**: **35 % de la nota final**.
    - Prueba escrita individual compuesta por análisis de algoritmos sobre redes (Dijkstra, Ford-Fulkerson, matching, eulerización, formulaciones TSP) y complejidad.
3. **Evaluación Continua de Laboratorios y Proyectos en Python**: **30 % de la nota final**.
    - Entrega periódica de cuadernos de laboratorio resueltos (15 %) y desarrollo de un proyecto práctico aplicado de modelado y optimización computacional (15 %).

Nota de superación: Para aprobar la asignatura por evaluación continua, el estudiante deberá obtener una calificación media ponderada igual o superior a **5.0 sobre 10**, con una nota mínima de **4.0 sobre 10** en cada uno de los dos exámenes parciales.

5.2. Convocatoria Extraordinaria

Los estudiantes que no hayan superado la asignatura en la convocatoria ordinaria dispondrán de una prueba extraordinaria global:

- **Examen Teórico-Práctico Final**: **70 % de la nota final** (evalúa los contenidos de los Bloques I y II).
- **Evaluación de Prácticas de Laboratorio**: **30 % de la nota final** (se conserva la nota obtenida durante el curso o se realiza una prueba práctica individual de programación en Python).

---

6 Bibliografía y recursos de referencia

6.1. Textos de referencia principal

- **Bazaraa, M. S., Sherali, H. D., & Shetty, C. M.** (2013). *Nonlinear Programming: Theory and Algorithms* (3rd ed.). John Wiley & Sons.
- **Boyd, S., & Vandenberghe, L.** (2004). *Convex Optimization*. Cambridge University Press.
- **Ahuja, R. K., Magnanti, T. L., & Orlin, J. B.** (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.
- **Bondy, J. A., & Murty, U. S. R.** (2008). *Graph Theory*. Springer Graduate Texts in Mathematics.
- **Nocedal, J., & Wright, S. J.** (2006). *Numerical Optimization* (2nd ed.). Springer Series in Operations Research.

6.2. Textos complementarios de algoritmos y computación

- **Diestel, R.** (2017). *Graph Theory* (5th ed.). Springer Graduate Texts in Mathematics.
- **Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C.** (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
- **Taha, H. A.** (2017). *Operations Research: An Introduction* (10th ed.). Pearson.
- **Vanderbei, R. J.** (2020). *Linear Programming: Foundations and Extensions* (5th ed.). Springer.

6.3. Recursos digitales y software

- **CVXPY Documentation**: Guía oficial y ejemplos de optimización convexa disciplinada (https://www.cvxpy.org/).
- **NetworkX Documentation**: Manual y tutoriales de análisis de redes y grafos en Python (https://networkx.org/).
- **SciPy Optimize Reference**: Documentación de algoritmos numéricos de optimización (https://docs.scipy.org/doc/scipy/reference/optimize.html).